
Campus Expense & Budget Tracker

Java Programming - Evaluated Course Project

VITyarthi | Project report | Version 1.0

Student name: Nirved Nitin Kamble

Registration number: 23BCE10439

Programme: Computer Science | Year: 4th Year

University: VIT Bhopal

Faculty / section: _____

Submission date: _____

Repository root URL: _____

Abstract

Campus Expense & Budget Tracker is a terminal-based Java application that helps a student record income and expenses, organize spending by category, and compare monthly spending with a planned budget. It uses Java classes, interfaces, collections, exception handling, date/time APIs, and file I/O. The implementation has no external Java dependencies and can be built and run without an IDE.

The application stores a complete ledger in a UTF-8 tab-separated file. It validates monetary amounts and calendar dates, rejects malformed stored data, and saves a new snapshot before changing its in-memory state. A menu supports ordinary use, while a command-line summary supports repeatable evaluation.

The supplied synthetic September dataset records INR 8,000.00 in income and INR 3,595.75 in expenses, leaving a monthly net of INR 4,404.25. The FOOD category exceeds its budget by INR 15.75. A dependency-free suite of 28 behavior tests checks calculations, validation, persistence, error handling, and CLI behavior.

Submission context

The provided instructions allow any project within the Java programming domain and require terminal execution, a public GitHub repository, a root README, and a structured report. This report follows a conventional academic structure because the course-specific report template and detailed marking rubric were not supplied. Faculty/section, submission date, and the published repository URL must be completed before upload.

1. Problem statement and objectives

Small daily expenses such as canteen meals, transport, printing, and campus events can be difficult to review when recorded in scattered notes. The project provides a consistent local record and a monthly view that distinguishes total spending from category-specific budget limits.

The objectives are to accept valid transactions, retain them across sessions, provide meaningful monthly totals, flag overspending, and demonstrate core Java concepts in code that a student can trace and modify.

2. Requirements and scope

Requirement	Implemented behavior
Record income and expenses	Date, type, category, positive INR amount, note, automatic ID
Review records	All records or one month, ordered by date and ID
Plan spending	One replaceable limit per category and month
Summarize a month	Income, expenses, monthly net, category remaining amount and status
Correct mistakes	Delete a transaction after typing YES, then add a corrected entry
Retain data	Automatic file save after every successful mutation
Support terminal evaluation	Non-interactive --summary command and dependency-free tests

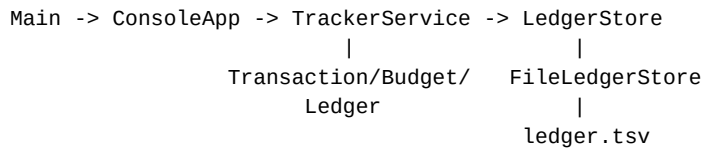
The user interface is a text menu. All amounts use INR. The six categories are FOOD, TRANSPORT, STUDY, HOSTEL, LEISURE, and OTHER. A missing budget is not treated as a zero limit. Budgets do not carry forward, and net for a month is not an account balance. This is a single-user local application with no bank connection.

3. Java concept mapping

Course concept	Evidence in the implementation
Classes and encapsulation	Transaction, Budget, and Ledger with private final fields
Interfaces and polymorphism	LedgerStore implemented by FileLedgerStore and test MemoryStore
Inheritance	DataFormatException extends IOException
Collections and generics	ArrayList, HashSet, EnumMap, typed lists and maps
Exceptions and resource management	Input validation, I/O errors, try-with-resources
File I/O	UTF-8 readers/writers, temporary file, snapshot replacement
Modular design	Separate model, service, storage, and CLI packages
Standard library use	BigDecimal, LocalDate, YearMonth, Comparator and lambdas

4. System design

The application separates data representation, business rules, file operations, and terminal presentation. Main manages process-level options and locking.



Component	Responsibility
Main	Parse arguments, open data directory, acquire lock, select menu or summary
ConsoleApp	Prompt for values, report input errors, print records and summaries
TrackerService	Add/delete transactions, replace budgets, calculate monthly results
Transaction and Budget	Store validated immutable domain values
Ledger	Hold a snapshot and reject duplicate IDs or budget keys
LedgerStore	Define load/save operations independent of storage implementation
FileLedgeStore	Parse and write one UTF-8 TSV snapshot

4.1 Design choices

Transactions use an enum for INCOME or EXPENSE because both types have identical fields and validation. Separate subclasses would not provide distinct behavior. An interface is useful for storage because production code and tests actually need different implementations.

Money is constructed from decimal strings using BigDecimal. The application does not use double for monetary values. Amount input is restricted to positive values with at most two decimal places. LocalDate validates real calendar dates, and YearMonth identifies the month without manual string slicing.

Immutable domain objects and defensive list copies prevent callers from changing the ledger unexpectedly. Each service mutation constructs a candidate snapshot. The file store saves that candidate before the service replaces its current reference. Failed saves therefore do not silently change the in-memory ledger.

4.2 Data format

The first TSV row contains CEBT1 and the next unused ID. T rows contain ID, date, type, category, amount, and note. B rows contain month, category, and limit. Tabs separate fields. Notes reject tabs and control characters, so no escaping parser is necessary. UTF-8 preserves Unicode notes. See DATA_FORMAT.md for the full schema, validation rules, and recovery steps.

5. Algorithms and error handling

5.1 Add transaction

1. Read and validate the date, type, category, amount, and note.
2. Create a Transaction with the stored next ID.
3. Copy the current transaction list and append the new object.
4. Construct a Ledger with the incremented next ID and existing budgets.
5. Save the candidate snapshot. Only after success, adopt it in memory.

Deleting a transaction follows the same save-before-commit sequence and retains the next-ID counter. Setting a budget replaces an existing matching month/category entry or appends a new one. IDs are not reused after deletion.

5.2 Monthly summary

Select transactions whose YearMonth equals the requested month. Sum INCOME and EXPENSE records separately. Monthly net is income minus expenses. Initialize an EnumMap with zero for each category and accumulate only expense amounts.

For a category with a budget, remaining amount equals limit minus spending. A negative remainder gives OVER BUDGET; zero gives AT LIMIT; a positive value gives WITHIN BUDGET. A category without a limit gives NOT SET. Income assigned to a category does not reduce that category's spending.

5.3 Persistence and errors

The store writes a complete temporary snapshot in the ledger's directory and closes it before replacement. It attempts an atomic move and falls back to an ordinary replacement if the filesystem does not support atomic moves. A file lock prevents simultaneous app sessions from writing the same data directory.

Invalid user input cancels the current action and returns to the menu. EOF exits without committing a partially entered transaction. Invalid ledger rows stop startup without overwriting the source file. A storage or lock failure returns exit status 1; invalid arguments return 2; normal completion returns 0.

The fallback replacement does not provide crash or power-loss durability. External manual edits are not synchronized with the app. These are documented limitations, and users should keep backups of important data.

5.4 Complexity

For n transactions and b budgets, loading, snapshot copying, and writing use $O(n+b)$ work and storage. A monthly list filters n rows and sorts k matches in $O(n+k \log k)$. Summary calculations currently reuse that sorted-list operation, giving $O(n+k \log k+b)$ complexity up to constant repeated passes. This is adequate for a small student ledger; a larger system could aggregate in one pass and use a database instead of rewriting the full file.

6. Testing and results

The test runner uses plain Java and throws `AssertionError` on a failed expectation. It does not require JUnit or the JVM's `-ea` option. Temporary test directories are cleaned after the run. Synthetic sample data contains no personal records.

Test area	Representative verified cases
Monetary validation	Exact 0.10 + 0.20, positive bounds, decimal precision, invalid text
Input and dates	Empty/control-character notes, invalid dates, leap date parsing
Calculations	Empty month, boundary dates, income exclusion, sorted lists
Budgets and IDs	Replacement, month isolation, deletion, stable next ID after reload
Failure handling	Failed add/delete/budget save retains earlier state
File storage	UTF-8 round trip, replacement, corrupt rows, duplicate keys
CLI integration	Scripted entry, restart, invalid arguments, EOF, deletion confirmation
Concurrency	Second session rejected while the directory lock is held

The complete suite contains 28 named behavior tests. Captured results and the exact validation environment are included in `TEST_RESULTS.txt`. The GitHub Actions workflow is configured for Windows and Linux; remote CI results require publication.

6.1 Sample demonstration

The sample contains September activity plus one record each in August and October. Those extra records make incorrect month filtering visible.

September result	Amount (INR)
Income	8000.00
Expenses	3595.75
Net for month	4404.25
FOOD spent / budget	215.75 / 200.00
FOOD remaining	-15.75
HOSTEL spent / budget	2500.00 / 2500.00

FOOD is OVER BUDGET and HOSTEL is AT LIMIT. TRANSPORT, STUDY, and LEISURE are WITHIN BUDGET. OTHER is NOT SET. The complete terminal output is supplied in `DEMO_OUTPUT.txt`, so the evaluator can compare actual output with expected values.

6.2 Reproduce

Run the platform-specific test command from `README.md`. After building, execute:

```
java -cp build/classes campus.Main --data sample-data --summary 2026-09
```

7. Installation and operation

Install a JDK, verify java and javac, and open a terminal in the repository root. The README provides the precise required version and commands for Windows, macOS, and Linux. No additional Java libraries, database, IDE, or account is needed to run the application. Build output is stored in build/.

The default data directory is data/. Use --data DIRECTORY to choose another location. A summary-only run adds --summary YYYY-MM. The interactive menu provides add, list, budget, summary, delete, and exit options. A successful mutation is saved immediately, so exiting does not require a separate save action.

8. Limitations and possible extensions

The project is intended for a single student's small local ledger. Data is plain text without encryption or authentication. It does not offer recurring entries, bank imports, currency conversion, budget removal, or direct transaction editing. A mistaken transaction is deleted and entered again. Monthly net has no opening balance. Budgets apply only to the selected month and category.

Useful extensions include note search, budget removal, a safe text-report export, and recurring expense reminders. Larger datasets would justify an indexed database and a single-pass summary algorithm. Each extension should add relevant tests and keep the documented behavior consistent with the source.

9. Conclusion

The implemented tracker solves a practical record-keeping problem with core Java. It connects object modeling, interface-based design, collections, exceptions, file I/O, and terminal interaction in a compact application. The sample data and repeatable tests make its behavior inspectable and suitable for a student code walkthrough.

10. References

- Oracle Java SE 8 BigDecimal API: <https://docs.oracle.com/javase/8/docs/api/java/math/BigDecimal.html>
- Oracle Java SE 8 Files API: <https://docs.oracle.com/javase/8/docs/api/java/nio/file/Files.html>
- Oracle Java SE 8 YearMonth API: <https://docs.oracle.com/javase/8/docs/api/java/time/YearMonth.html>
- Eclipse Temurin installation: <https://adoptium.net/installation>
- VITyarthi submission instructions supplied in the accompanying conversation.

Assistance and student review

This project package was prepared with AI assistance. The student should review, run, understand, and customize the work, and disclose assistance as required by the course. No claim of sole authorship or fabricated student identity is made. Before submission, complete the cover details, check the actual course rubric and report format, and publish the repository with public visibility.